I can create a PDF for you containing the Node.js + Express + Mongoose study checklist for building a notes app. Here's the content formatted for PDF:

Node.js + Express + Mongoose Notes App Study Checklist

1️⃣ Node.js Basics

- Installing Node and npm
- Creating a project: `npm init -y`
- Running a server: `node server.js`
- Modules: `import` / `require`
- `process.env` and environment variables
- Using `console.log` for debugging
- Async/await basics

2️⃣ Express.js Essentials

- Installing Express: `npm install express`
- Creating a server: `const app = express()`
- Defining routes:
- `app.get("/")` for GET requests
- `app.post("/add")` for POST requests
- Middleware:
- `express.json()` for parsing JSON
- `express.urlencoded({ extended: true })` for form data
- `express.static("public")` for serving HTML/CSS/JS
- Sending responses:
- `res.send()`
- `res.json()`
- `res.render()` (if using EJS)
- Redirects: `res.redirect("/")`

3️⃣ Mongoose & MongoDB

- Installing Mongoose: `npm install mongoose`
- Connecting to MongoDB:
- Local: `mongodb://127.0.0.1:27017/dbname`
- Atlas: cloud connection string
- Creating a schema and model:

```
const noteSchema = new mongoose.Schema({
  username: String,
  heading: String,
  content: String
});
const Note = mongoose.model("Note", noteSchema);
```

- CRUD operations:
- `Note.find()` → read all notes
- `Note.findById()` → read one note
- `new Note({...}).save()` → create
- `Note.updateOne()` → update
- `Note.deleteOne()` → delete

4 Frontend JavaScript (DOM + Fetch API)

- Selecting elements: `document.getElementById()`, `querySelector()`
- Creating elements dynamically: `document.createElement()`
- Modifying content: `element.textContent` or `innerHTML`
- Appending elements: `parent.appendChild()`
- Loops with `forEach()` to render lists from an array (like notes)
- Handling events:
- `element.addEventListener("click" / "submit", callback)`
- `e.preventDefault()` for forms
- Fetch API:
- `fetch("/api/notes")` → GET
- `fetch("/api/notes", { method: "POST", headers: {...}, body: JSON.stringify(...) })`
- Async/await for handling responses

5 HTML & Forms

- Basic HTML structure: `<html><head><body>`
- Forms:
- `<input>` for text fields
- `<textarea>` for content
- `<button type="submit">`
- Linking JS & CSS: `<script src="app.js"></script>` and `<link rel="stylesheet" href="style.css">`
- IDs and classes for selecting elements in JS

6 CSS (optional for styling)

- Basic layout
- Styling lists (`ul` / `li`)

- Form styling
- Optional: Flexbox/Grid for layout

7️⃣ Environment Variables (.env)

- Why use `.env` for:
- `PORT`
- `MONGO_URI`
- Loading with `dotenv` package

Recommended Order to Study

1. Node.js basics → npm, modules, server
2. Express routes + middleware
3. MongoDB basics → collections, documents
4. Mongoose schemas and CRUD
5. Frontend JS → DOM, events, fetch
6. Connect frontend → backend (fetch API)
7. Optional: HTML/CSS polish and deployment

This cheat sheet covers everything you need to build a notes app from scratch with Node.js, Express, and Mongoose, including the frontend JS for displaying and submitting notes.